这份文档是专门为你准备的 **“喂给 Copilot / 大语言模型的仓库检索与参考指南”**。你可以直接将以下方框内的全部内容复制给 Copilot，它会指导 Copilot 精准定位到你需要的开源代码库、API 文档以及架构蓝图，从而为你生成正确的毕设代码。

复制以下内容发送给 Copilot 👇

【System Prompt: 你的任务与角色】你现在是我的高级研发辅助工程师（Copilot）。我正在进行一项名为《面向分布式推理的零知识证明框架设计与低开销优化》的毕业设计。我的核心架构是基于 **DSperse 框架** 提出的“模型切片（Model Slicing）”与“定向验证（Targeted Verification）”理念1, 2。我需要用 Python 编写一个 Master-Worker 架构的分布式系统，并利用现成的 ZKML 编译器为各个模型切片生成局部的零知识证明。请你仔细阅读以下我为你提供的**核心开源仓库（Repo）、官方文档链接及参考资料**。在接下来的代码生成中，你必须基于这些资料的 API 和设计模式来为我编写代码。

【核心 Repo 1：底层的零知识证明引擎 EZKL】我们需要使用 EZKL 将 PyTorch 导出的 ONNX 模型转化为 Halo2 零知识证明电路，并生成证明3-5。

1. **官方开发文档 (重点阅读)**: https://docs.ezkl.xyz/ 6-8
2. **代码仓库与示例**: https://github.com/zkonduit/ezkl
3. **你需要学习的 API**: 请你检索并学习 EZKL Python API 的标准工作流，特别是以下函数的用法：
4. ezkl.gen_settings()：为子模型生成电路配置9。
5. ezkl.compile_circuit()：编译零知识证明电路9。
6. ezkl.prove() 和 ezkl.verify()：用于 Worker 节点生成证明以及 Master 节点验证证明5。
7. **注意**：在生成代码时，请特别注意 EZKL 对浮点数的量化（Quantization）处理参数（如 scale），我们需要使用对称整数（INT8或INT16）量化10。

【核心 Repo 2：模型切片与电路设计蓝图 DSperse】我的分布式切片架构直接参考自 DSperse 前沿框架。请你检索并理解其架构设计逻辑：

- **架构概念论文**: https://arxiv.org/html/2508.06972v3 或 https://arxiv.org/abs/2508.06972 7, 11
- **开源蓝图与电路参考**: https://github.com/inference-labs-inc/zkml-blueprints 12, 13
- **你需要学习的逻辑**: DSperse 框架将大型神经网络沿层（Layers）边界切分成离散的子块（Slices），分配给多个节点，独立生成证明14, 15。请你在为我编写 PyTorch 模型切片代码（如切分 LeNet 或 ResNet）时，确保切片间没有反向依赖，并能各自独立导出为 .onnx 文件16。

【补充 Repo 3：大型模型分布式 ZKP 参考 (zkLLM)】如果我们的实验需要扩展到更复杂的非线性层，或者参考大模型的高效 ZKP 实现，请查阅 zkLLM 的代码实现：

- **开源仓库**: https://github.com/jvhs0706/zkllm-ccs2024 17
- **参考价值**: 了解如何使用并行化的查找表（tlookup）和状态承诺来优化复杂的激活函数和非线性计算18, 19。

【Copilot 检索与执行指令】我已经为你提供了相关的链接和背景。现在，请你执行以下操作：

1. **联网检索（如果支持）**：请访问 https://docs.ezkl.xyz/ 8 查阅最新的 EZKL Python API 示例代码（比如从 ONNX 导出到生成 Proof 的完整 Jupyter Notebook 流程）。
2. **确认理解**：请向我简要复述你对“利用 EZKL 对 ONNX 模型进行零知识证明”流程的理解。
3. **准备编码**：一旦你确认理解完毕，我将让你开始编写 **“第一阶段：单机跑通 EZKL 的 Python 脚本”**。请告诉我你是否已准备好。

使用建议：

1. Copilot 或 ChatGPT 看到这个 Prompt 后，会立即去抓取 docs.ezkl.xyz 的最新 Python 语法5, 8，这能有效避免它产生幻觉或使用过时的旧版 API。
2. 附带的 zkml-blueprints 仓库链接12会让 AI 明白你需要的不仅仅是跑通代码，而是要像 DSperse 论文里那样对模型进行**层级切割**。